

Binary Tree Search Experiment

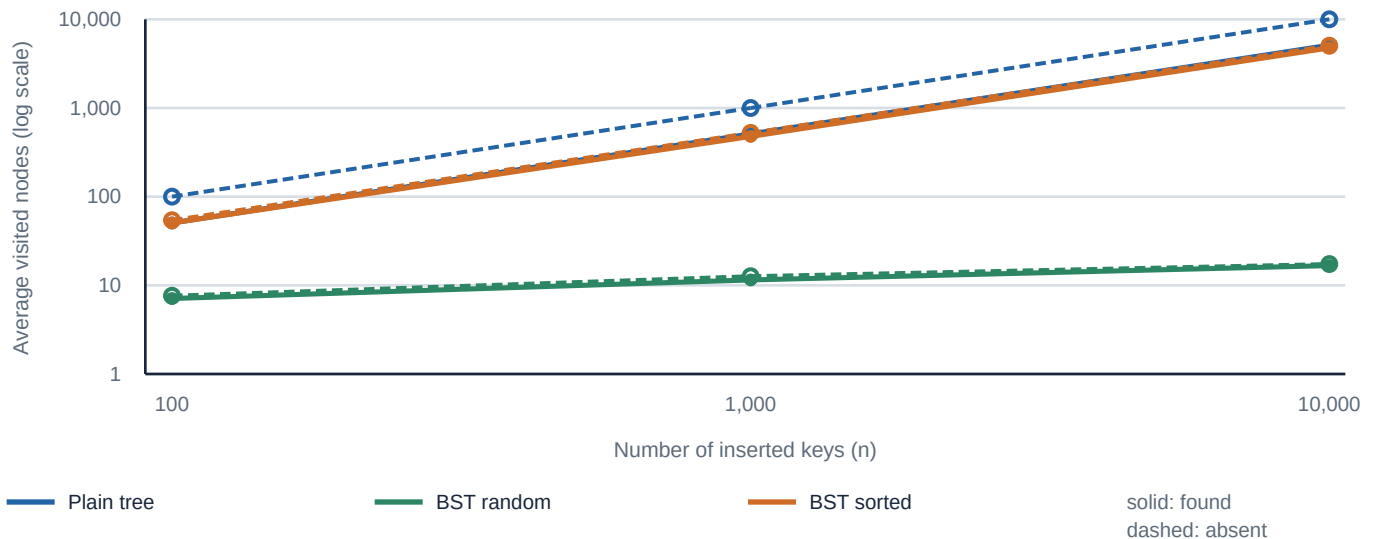
Level-order binary tree vs. random and sorted binary search trees

Method: seed 42; 100 found and 100 absent queries per size; same queries for all three trees.
Unique random even keys; absent keys are random odd values from the same numeric range.

Average nodes visited per search

n	Tree	Found	Absent
100	Plain binary tree	50.50	100.00
	BST - random order	7.10	7.63
	BST - sorted order	50.50	54.25
1,000	Plain binary tree	510.28	1,000.00
	BST - random order	11.50	12.67
	BST - sorted order	477.63	524.38
10,000	Plain binary tree	5,106.19	10,000.00
	BST - random order	16.68	17.38
	BST - sorted order	4,759.51	5,043.70

Visited nodes scale sharply when sorted insertion makes a BST a chain



Why the results differ

Plain tree: Its nodes have no key ordering, so level-order search checks nodes one by one. A missing key visits all n nodes; a found key usually visits about half.

Random BST: Each comparison chooses one subtree. Random insertion keeps paths relatively short here, giving roughly 7-17 visits, including absent searches.

Sorted BST: Every new key becomes the right child of the previous maximum. The tree becomes a chain, and search cost rises toward n. The plain tree stays complete by levels because insertion always fills the earliest open child slot in its queue, left before right.